

Milestone 3 Report

Smart MCQ Solver Challenge — RAG, FAISS, Cross-Encoders & Adversarial Retrieval

This milestone builds a full Retrieval-Augmented Generation (RAG) pipeline on top of the knowledge base constructed from train.csv correct answers. It covers vector search with FAISS, two-stage retrieval with cross-encoder reranking, token budget analysis, and the adversarial failure mode where bad retrieval actively hurts model performance.

Knowledge Base & FAISS Index Setup

The knowledge base is built by iterating over every row in train.csv and storing only the text of the correct answer option (kb.append(str(row[correct_letter]))). This means the KB contains 2000 documents, one per training question, each being the text of the ground-truth correct option — not the question itself.

The KB is then embedded with all-MiniLM-L6-v2 and stored in a FAISS IndexFlatL2 (exact L2-distance search, no approximation). At query time, the prompt is embedded with the same model and the index returns the k nearest documents by L2 distance in embedding space.

↳ IndexFlatL2 does exact nearest-neighbour search — no approximation. Fine for 2000 documents; for millions of documents you'd switch to IndexIVFFlat or HNSW for speed.

Q1. Zero-shot probability for the correct option at row 150

Ran facebook/bart-large-mnli zero-shot classifier on the row 150 prompt with all 5 option texts as candidate_labels. BART-MNLI frames each candidate as a hypothesis and scores entailment probability via softmax across the 5 labels. Extracted the score assigned specifically to the text of the ground-truth correct option (matched by exact string) and rounded to 3 decimal places.

↳ With 5 candidates and a softmax, scores sum to ~1.0 and the model has no external context — it's relying entirely on its pre-trained knowledge to judge which option the prompt entails.

Q2. FAISS retrieval rank of the true document for row 150

Embedded the row 150 prompt with all-MiniLM-L6-v2 and queried the FAISS index for k=10 nearest documents. The true document (kb[150], the correct option text for row 150) may or may not appear at rank 1 — FAISS is searching for semantically similar correct-answer texts from *other* questions, so it has no guarantee of finding the one from the same row.

↳ This is the fundamental tension in this KB design: the prompt is a question, but the KB contains answer texts. The bi-encoder embeds them in the same space but the prompt and its own correct answer may not be nearest neighbours — other questions' answers with similar wording could rank higher.

Q3. Cross-encoder reranking rank of the true document

Took the 10 documents from Q2, paired each with prompt_150 as [prompt, doc], and scored all 10 pairs with cross-encoder/ms-marco-MiniLM-L-6-v2. Sorted by cross-encoder score descending and found the new rank of kb[150]. The cross-encoder attends jointly to both the prompt and document tokens, giving it much more context to judge relevance than the bi-encoder's separate embeddings.

↳ ms-marco-MiniLM-L-6-v2 was trained on the MS MARCO passage retrieval dataset — it's designed exactly for this: scoring (query, passage) pairs by relevance. It doesn't produce a probability, just a raw score; higher = more relevant.

Q4. Token count for RAG string at row 42 with top k=5 documents

Retrieved top-5 documents for row 42's prompt from FAISS. Concatenated the 5 document strings with a single space, then built the RAG string as exactly: 'Context: [concatenated_docs] Question: [prompt]'. Tokenized this full string with bert-base-uncased using truncation=False and counted len(input_ids).

↳ Not truncating is important here — the question asks for the true total token count, not the truncated version. In a real RAG system this number matters a lot: BERT's max is 512 tokens, so a RAG string with 5 long documents can easily overflow and force truncation that cuts off the actual question.

Q5. RAG with true document — zero-shot probability change for row 150

Used kb[150] (the actual correct option text for row 150) as the injected context, built the RAG string as 'Context: [true_doc] Question: [prompt]', and re-ran the same zero-shot classifier with the same 5 candidate labels. Compared the correct option's probability before context (Q1) and after adding the true context.

↳ This is the best-case RAG scenario: perfect retrieval. If the correct option's probability increases substantially, it confirms that injecting the right context genuinely helps the model. If the change is small, it suggests the model's internal knowledge was already sufficient for this question.

Q6. Adversarial RAG — forced wrong context (KB index 999)

Took kb[999] — a completely unrelated fact from a different question — and injected it as context: 'Context: [kb[999]] Question: [prompt_150]'. Ran the same zero-shot classifier. Compared the correct option's probability against Q1 (no context) and Q5 (true context).

↳ This demonstrates the 'Garbage In, Garbage Out' failure mode. BART-MNLI is trained to treat context as ground truth — if the injected context supports a wrong option, the model will often shift probability toward that wrong option even when its internal knowledge points elsewhere. Bad retrieval can actively make things worse than no retrieval at all.

Q7. Hit Rate for first 100 rows — top k=5 FAISS retrieval

For each of the first 100 rows, embedded the prompt, retrieved top-5 KB documents, and checked whether the exact text of the correct option appears in any of those 5 documents (using Python's 'in' operator for substring containment). Hit Rate = (number of hits / 100) * 100, rounded to 1 decimal place.

↳ This is a recall metric: what fraction of the time does the retrieval step find the right document at all? A low hit rate here means even a perfect reranker can't fix the pipeline — if the correct document isn't in the top-5, the cross-encoder never gets a chance to rank it first.

Q8. Full RAG pipeline MAP@3 across first 20 rows

For each of the first 20 rows, ran the complete pipeline: (1) FAISS retrieval k=5, (2) cross-encoder reranking — pick single best document, (3) build RAG string 'Context: [best_doc] Question: [prompt]', (4) zero-shot classify with all 5 option texts as labels, (5) sort options by score descending, take top-3 letter labels, (6) compute MAP@3 against the true answer. Final score = mean MAP@3 across all 20 rows, rounded to 3 decimal places.

↳ The label-to-letter mapping requires matching zero-shot output labels (which are option texts) back to the option letter (A-E) — the classifier returns label strings not letters, so this mapping step is required before computing MAP@3.

The three Q1/Q5/Q6 scores together tell the clearest story: no-context vs. true-context vs. adversarial-context. If true context substantially lifts the correct option's probability and adversarial context drops it below no-context, that's direct evidence that retrieval quality is the binding constraint in this RAG pipeline — more than the classifier or the reranker.